4. Scheduling

Computer and Virtual Technologies Engineering
Software Engineering

Operating Systems

2025-2026

Universidad LOYOLA

Contents

1) Introduction
2) Types of processor scheduling
3) Scheduling algorithms
   - FCFS: First-Come-First-Served
   - Round Robin
   - SPN: Shortest Process Next
   - SRT: Shortest Remaining Time
   - HRRN: Highest Response Ratio Next
   - Feedback

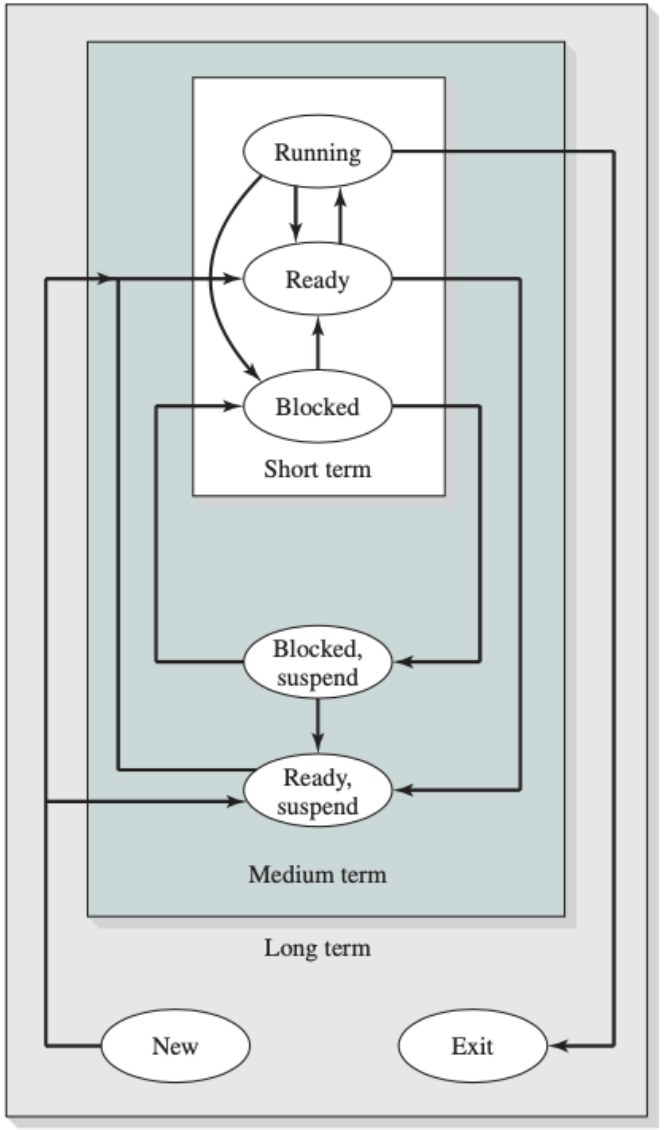
Introduction

- The aim of processor **scheduling** is to **assign processes to be executed** by the processor or processors over time, in a way that meets system objectives, such as **response time**, **throughput** and **processor efficiency**. This chapter is dedicated to the assignment of processor time to each process.
- The decision of **which process (and how long)** to execute is taken by the **OS**.
- The criteria for the use of the CPU and the different **scheduling algorithms** will be described.
- Scheduling affects the **performance** of the system because it determines which processes will wait and which will progress.

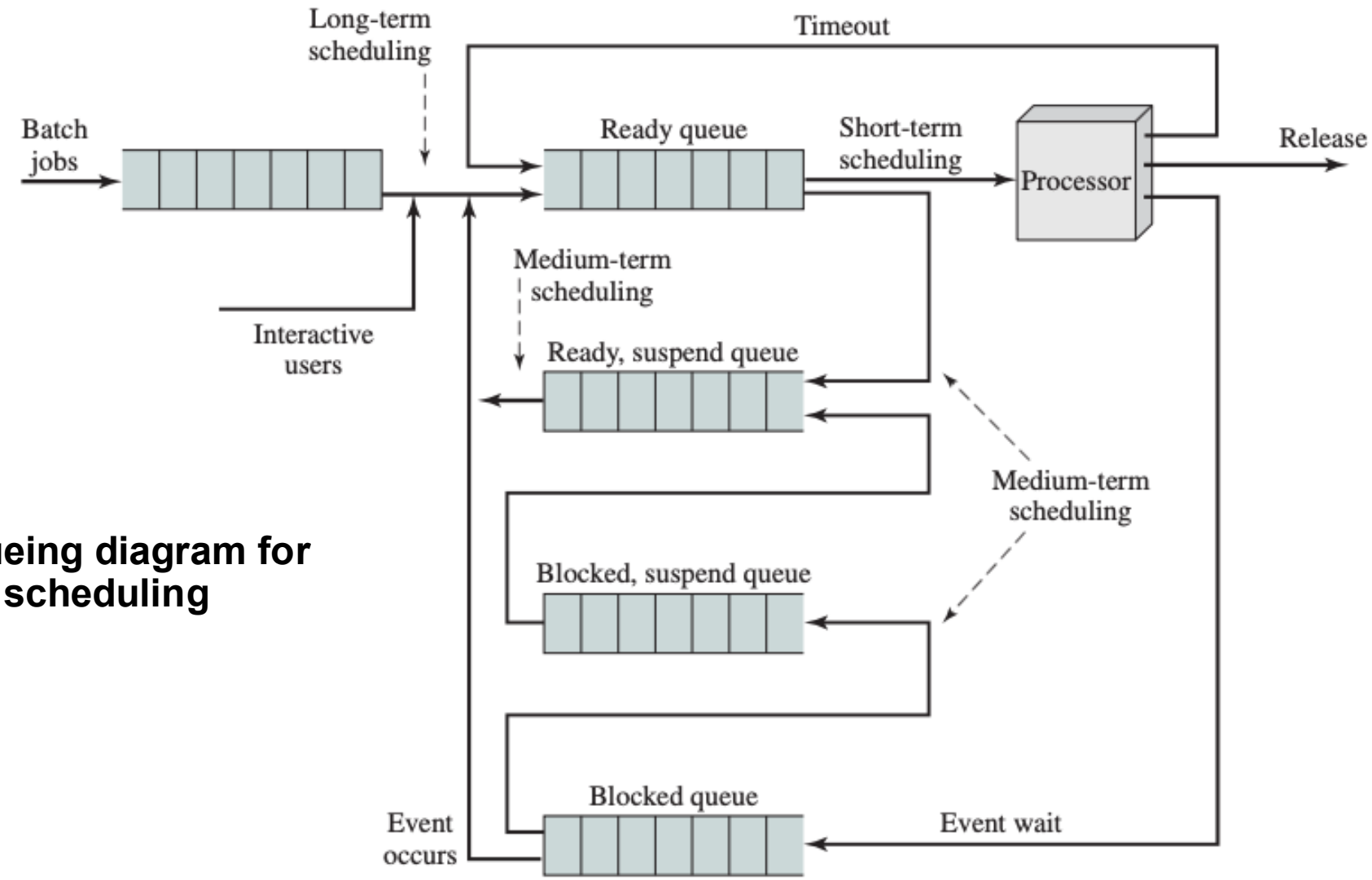
Types of processor scheduling

Four types of scheduling are typically involved:

- **Long-term scheduling:** The decision to add to the pool of processes to be executed.
- **Medium-term scheduling:** The decision to add to the number of processes that are partially or fully in main memory.
- **Short-term scheduling:** which available process will be executed by the processor.
- **I/O scheduling:** which process's pending I/O request shall be handled by an available I/O device.



Types of processor scheduling



Queueing diagram for scheduling

Types of processor scheduling

Long-term scheduling

- The long-term scheduler determines which programs are admitted to the system for processing.
- The decision to create a new process is generally driven by the desired degree of multiprogramming. The more processes that are created, the smaller is the percentage of time that each process can be executed.
- The long-term scheduler may limit the degree of multiprogramming to provide satisfactory service to the current set of processes.
- Each time a job terminates, the scheduler may decide to add one or more new jobs.

Types of processor scheduling

Medium-term scheduling

- Medium-term scheduling is part of the swapping function.

Types of processor scheduling

Short-term scheduling

- The short-term scheduler, also known as the dispatcher, executes most frequently and makes the fine-grained decision of which process to execute next.
- It is invoked whenever an event occurs that may lead to the blocking of the current process. Examples of such events:
  - Clock interrupts.
  - I/O interrupts.
  - Operating system calls.
  - Signals (e.g., semaphores).

Scheduling algorithms

Short-term scheduling criteria

- The main objective of the short-term scheduling is to allocate processor time to optimize one or more aspects of system behavior.
- Criteria can be categorized along:
  - User-oriented criteria.
  - System-oriented criteria.
- Another dimension to classify criteria:
  - Performance related.
  - Not directly performance related.

Scheduling algorithms: short-term criteria

User Oriented, Performance Related

Turnaround time This is the interval of time between the submission of a process and its completion. Includes actual execution time plus time spent waiting for resources, including the processor. This is an appropriate measure for a batch job.

Response time For an interactive process, this is the time from the submission of a request until the response begins to be received. Often a process can begin producing some output to the user while continuing to process the request. Thus, this is a better measure than turnaround time from the user's point of view. The scheduling discipline should attempt to achieve low response time and to maximize the number of interactive users receiving acceptable response time.

Deadlines When process completion deadlines can be specified, the scheduling discipline should subordinate other goals to that of maximizing the percentage of deadlines met.

User Oriented, Other

Predictability A given job should run in about the same amount of time and at about the same cost regardless of the load on the system. A wide variation in response time or turnaround time is distracting to users. It may signal a wide swing in system workloads or the need for system tuning to cure instabilities.

System Oriented, Performance Related

Throughput The scheduling policy should attempt to maximize the number of processes completed per unit of time. This is a measure of how much work is being performed. This clearly depends on the average length of a process but is also influenced by the scheduling policy, which may affect utilization.

Processor utilization This is the percentage of time that the processor is busy. For an expensive shared system, this is a significant criterion. In single-user systems and in some other systems, such as real-time systems, this criterion is less important than some of the others.

System Oriented, Other

Fairness In the absence of guidance from the user or other system-supplied guidance, processes should be treated the same, and no process should suffer starvation.

Enforcing priorities When processes are assigned priorities, the scheduling policy should favor higher-priority processes.

Balancing resources The scheduling policy should keep the resources of the system busy. Processes that will underutilize stressed resources should be favored. This criterion also involves medium-term and long-term scheduling.

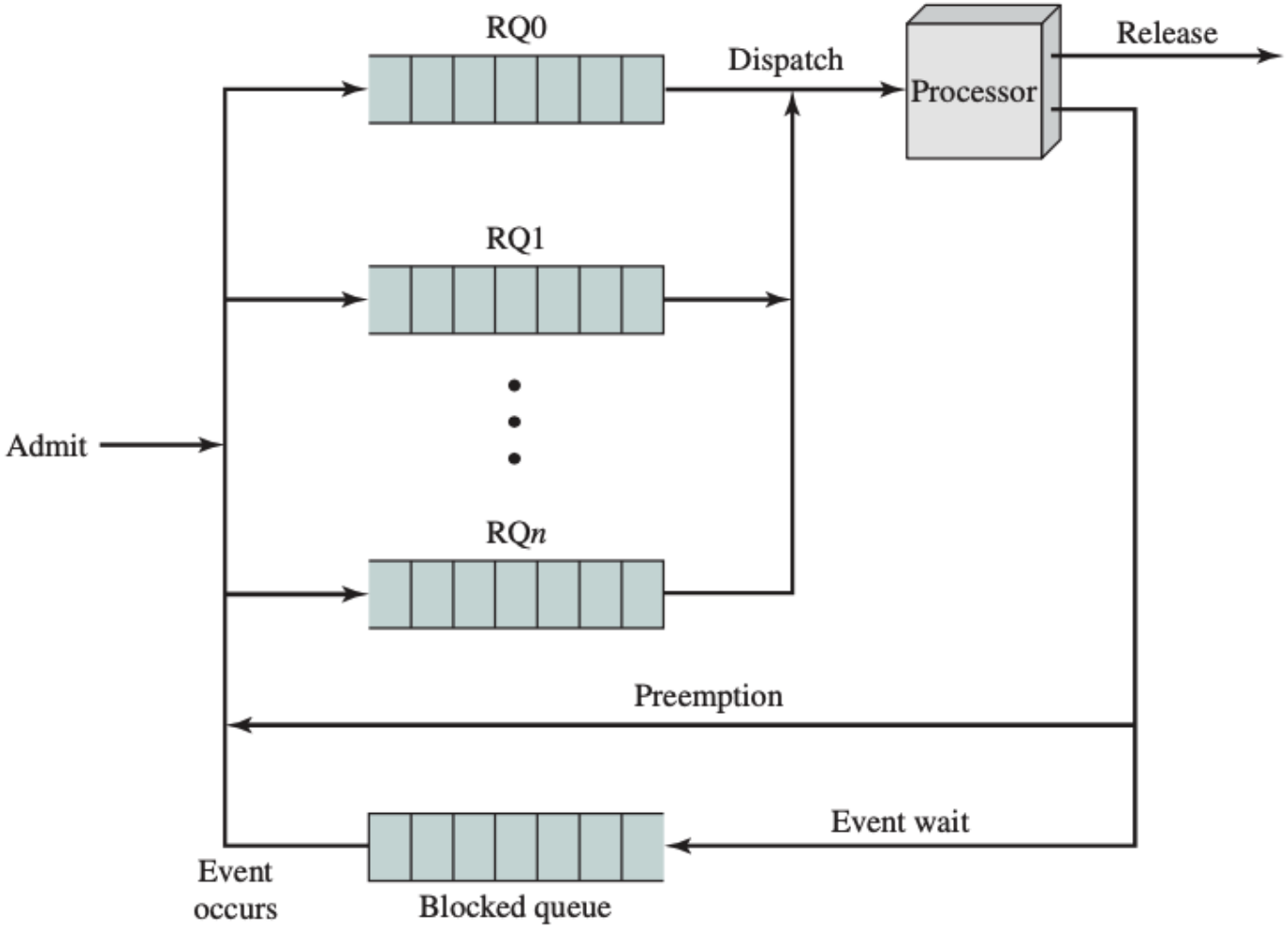
Scheduling algorithms

Decision mode. There are two general categories:

- **Nonpreemptive:** once a process is in the Running state, it continues to execute until it terminates or it blocks itself to wait for I/O or to request some OS service.
- **Preemptive:** the currently running process may be interrupted and moved to the Ready state by the OS. The decision to preempt may be performed when a new process arrives, when an interrupt occurs that places a blocked process in the Ready state, or periodically, based on clock interrupt.

Scheduling algorithms: the use of priorities

- Each process is assigned a priority.
- RQ0 higher priority.
- If RQ0 is empty, then RQ1 is examined, and so on.
- If there are on or more processes in the queue, a process is selected using some scheduling policy.
- Lower priority process may suffer starvation.



Scheduling algorithms: FCFS

- FCFS: First-Come-First-Served, also know as First-In, First-Out (FIFO).
- When the currently running process ceases to execute, the process that has been in the ready queue the longest is selected for running.
- FCFS performs much better for long process that short ones.

Scheduling algorithms: FCFS

- FCFS: First-Come-First-Served, also know as First-In, First-Out (FIFO).
- When the currently running process ceases to execute, the process that has been in the ready queue the longest is selected for running.
- FCFS performs much better for long process that short ones. **Example**:

Process	Arrival Time	Service Time (T_s)	Start Time	Finish Time	Turnaround Time (T_r)	T_r/T_s
W	0	1	0	1	1	1
X	1	100	1	101	100	1
Y	2	1	101	102	100	100
Z	3	100	102	202	199	1.99
Mean					100	26

The normalized turnaround time (Tr/Ts) for Y is 100 times the required processing time. This will happen whenever a short process arrives just after a long process.

Note: turnaround time is the total time in the system (Finish_Time - Arrival_Time).

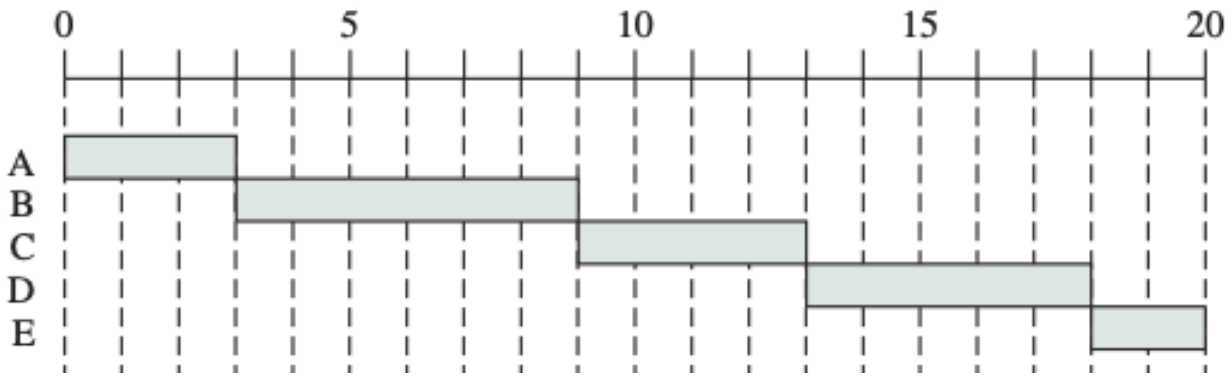
Scheduling algorithms: FCFS

- FCFS: First-Come-First-Served, also know as First-In, First-Out (FIFO).
- When the currently running process ceases to execute, the process that has been in the ready queue the longest is selected for running.
- FCFS performs much better for long process that short ones.
- It is **combined with a priority scheme** to provide an effective scheduler.

Scheduling algorithms: FCFS

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

First-come-first served (FCFS)



Scheduling algorithms: Round Robin

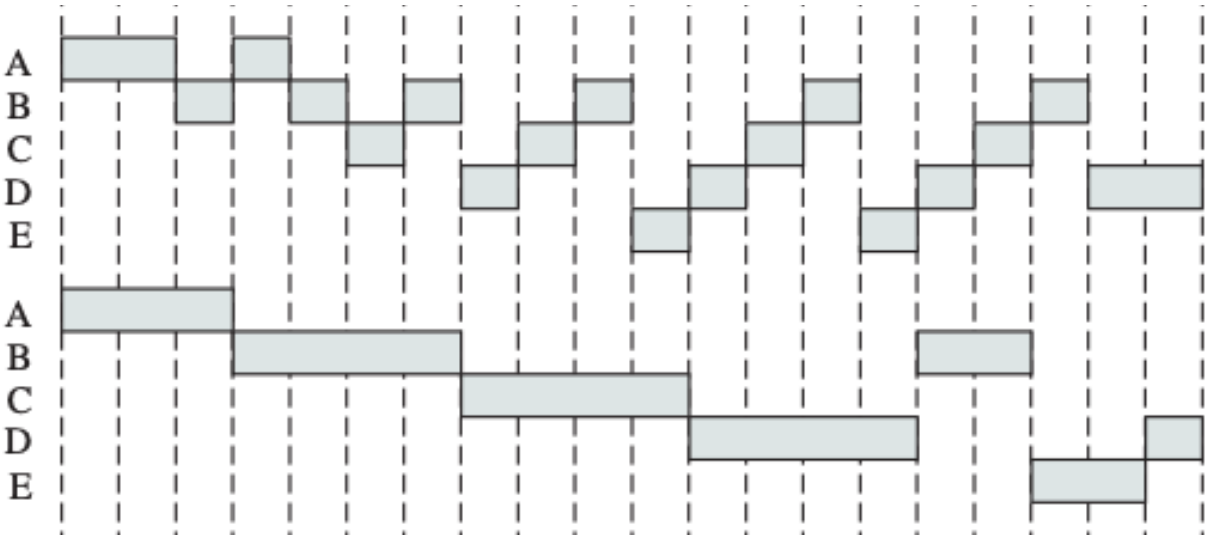
- A straightforward way to reduce the penalty that short jobs suffer with FCFS is to use preemption based on a clock. **Round robin** is the simplest way to do it.
- A clock interrupt is generated at periodic intervals. When the **interrupts occurs**, the **currently running process is placed in the ready queue**, and the **next ready job is selected on a FCFS basis.**
- This technique is also known as **time slicing** (each process is given a slice of time before being preempted).
- The principal design issue is the time quantum:
  - Useful guide: slightly greater than the time required for process function.
  - Less: most processes will require at least two time quanta.
  - Limiting case of a time quantum that is longer than the longest running process: Round Robin degenerates to FCFS.

Scheduling algorithms: Round Robin

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Round-robin (RR), $q = 1$

Round-robin (RR), $q = 4$



Scheduling algorithms: Shortest Process Next

- Another approach to reducing the bias in favor of long processes is the Shortest Process Next (SPN) policy.
- This is a **nonpreemptive** policy in which the **process with the shortest expected processing time is selected next.**
- It is necessary to know the required processing time of each process (or **at least estimate it**).
- Risk of starvation for longer processes, as long as there is a steady supply of shorter processes.
- Operating system may keep a running average of each *burst* for each process*:*

$$S_{n+1} = \frac{1}{n}\sum_{i=1}^{n} T_i$$

T: processor execution time
S: predicted value

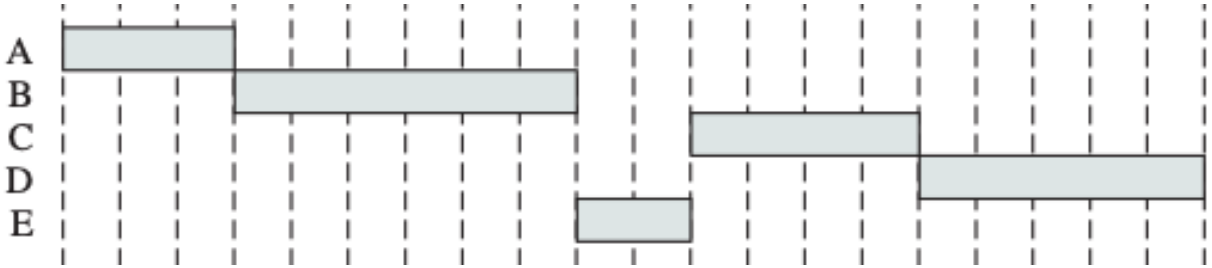
To avoid recalculating the entire summation each time: $S_{n+1} = \frac{1}{n}T_n + \frac{n-1}{n}S_n$

Greater weight to more recent instances,
exponential averaging (time series of past values): $S_{n+1} = \alpha T_n + (1-\alpha)S_n$ $\quad 0 < \alpha < 1$

Scheduling algorithms: Shortest Process Next

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Shortest process next (SPN)



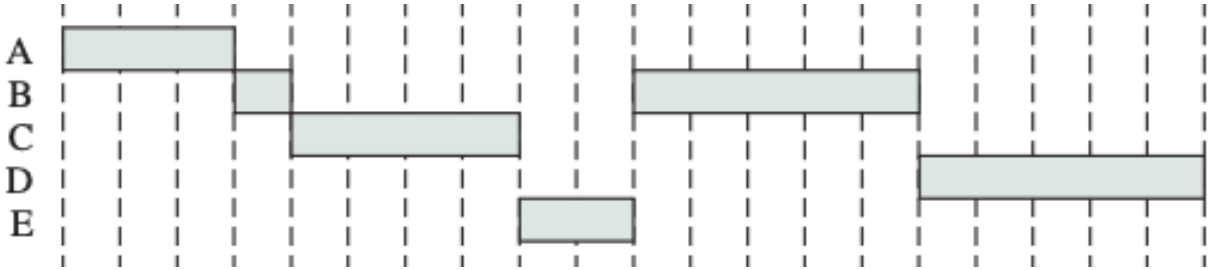
Scheduling algorithms: Shortest Remaining Time

- The Shortest Remaining Time (SRT) is a **preemptive version of SPN**.
- The scheduler chooses the process that has the shortest expected remaining processing time.
- The scheduler must have an estimate of processing time to perform the selection.
- There is a risk of starvation of longer processes.
- Unlike round robin, no additional interrupts are generated, reducing overhead.

Scheduling algorithms: Shortest Remaining Time

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Shortest remaining time (SRT)



Scheduling algorithms: Highest Response Ratio Next

- Consider the following ratio:

$$R = \frac{w + s}{s}$$

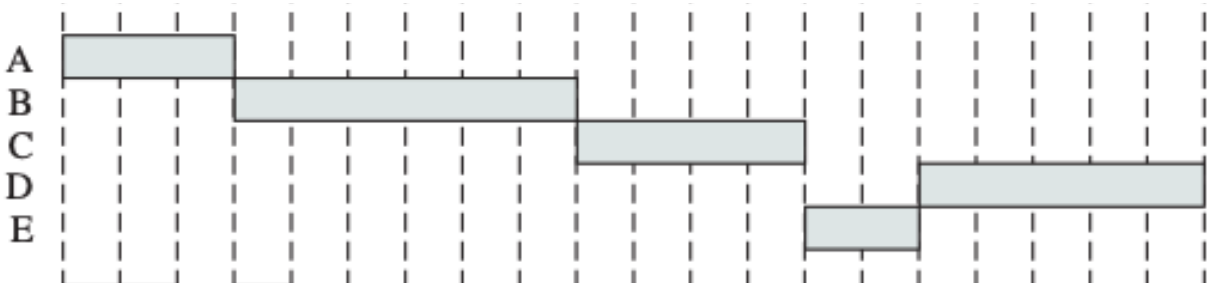
R: response ratio
w: time spent waiting for the processor
s: expected service time.

- Scheduling rule: when the current process completes or is blocked (**nonpreemptive**), choose the ready process with the greatest value of R.
- This approach is attractive because it accounts for the age of the process.
- **Shorter jobs are favored** (a smaller denominator yields a larger ratio).
- Aging without service increases also the ratio: **longer process will eventually get past competing shorter jobs.**
- Expected service time must be estimated in HRRN.

Scheduling algorithms: Highest Response Ratio Next

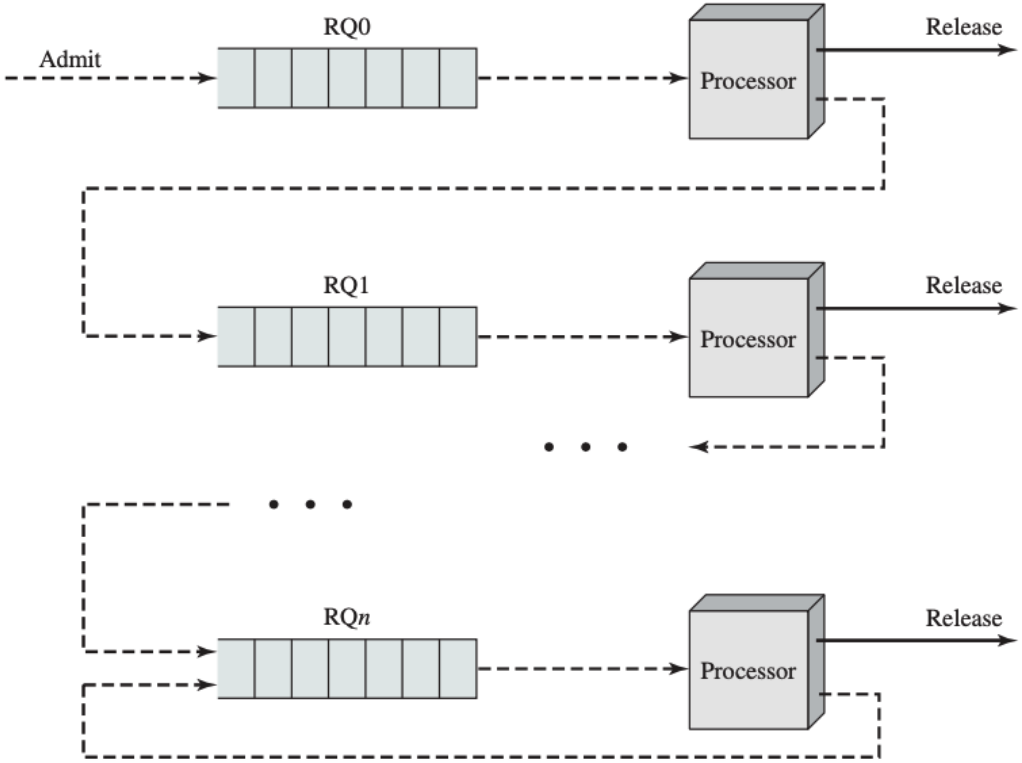
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Highest response ratio next (HRRN)



Scheduling algorithms: Feedback

- Another way of establishing a preference for shorter jobs is to **penalize jobs that have been running longer.**
- **Preemptive scheduling (time quantum) and a dynamic priority mechanism.**
- When a process first enters the system, it is placed in RQ0. After its first preemption, when it returns to the Ready state, it is placed in RQ1. Each subsequent time it is moved to the next lower priority queue.
- Short process will complete quickly.
- Longer processes will gradually drift downwards.
- Within each queue, a FCFS mechanism is used.
- Once in the lowest priority queue, a process cannot go lower, but is returned to this queue until it completes execution.
- This is known as a **multilevel feedback**.

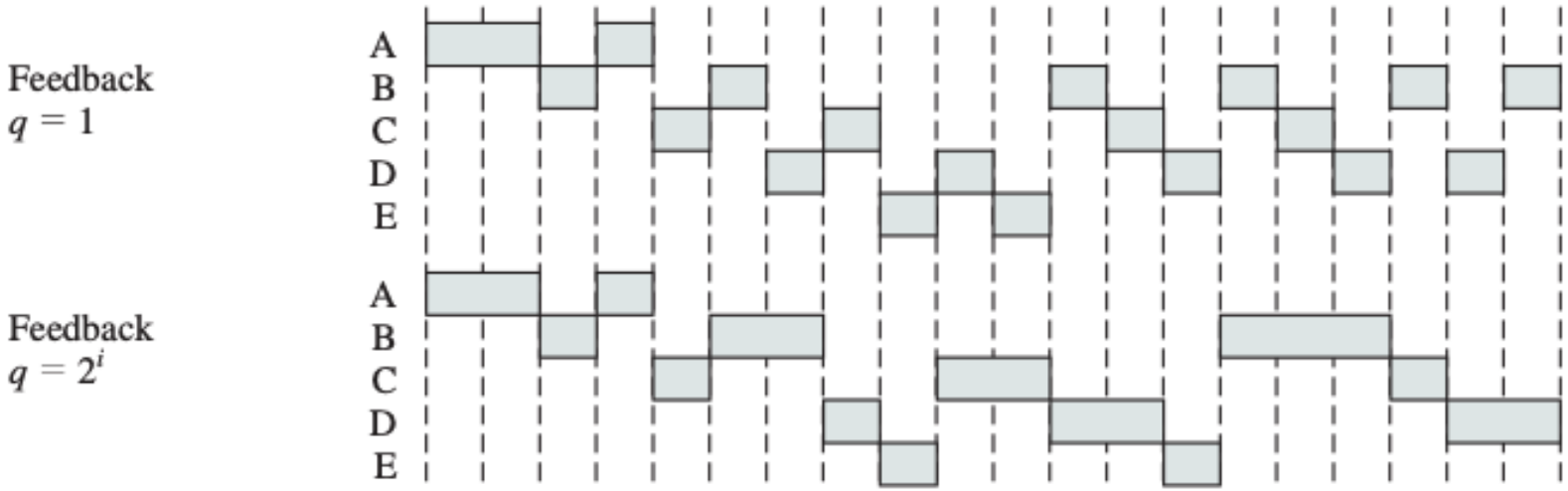


Scheduling algorithms: Feedback

- Variations: perform preemption in the same fashion as for round robin at periodic intervals.
- To avoid starvation when new process enters the system: varying the preemption times according to the queues.
- A process from RQ0 queue is allowed to execute for one time unit.
- A process from RQ1 queue is allowed to execute for two time units, and so on.
- In general, for RQi is allowed to executed 2^i time units before preemption.

Scheduling algorithms: Feedback

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2



Universidad
LOYOLA

Universidad
LOYOLA